Aarya Arban

T11 – 05

Assignment No. 4

Aim: To apply navigation and routing in Flutter Apps.

Navigation and Routing in Flutter

Navigation and routing in Flutter allow users to move between different screens (pages) in an app. Flutter provides two main approaches for navigation:

1. **Basic Navigation using Navigator.push() and Navigator.pop()**

2. **Named Routes for structured navigation management**

---

1. Basic Navigation (Push & Pop)

Flutter's Navigator manages a stack-based navigation system. When a new screen is pushed onto the stack, it appears, and when popped, it is removed.

Navigating to a New Screen

```
Navigator.push(
  context,
  MaterialPageRoute(builder: (context) => SecondScreen()),
);
```

This moves from the current screen to SecondScreen().

Going Back to the Previous Screen

```
Navigator.pop(context);
```

This removes the current screen and returns to the previous one.

---

2. Named Routes (Recommended for Large Apps)

Instead of defining routes manually using MaterialPageRoute, we can use **named routes** in MaterialApp.

Step 1: Define Routes in main.dart

```
void main() {
  runApp(MaterialApp(
    initialRoute: '/',
    routes: {
      '/': (context) => HomeScreen(),
      '/second': (context) => SecondScreen(),
    },
  ));
}
```

Step 2: Navigate Using Named Routes

```
Navigator.pushNamed(context, '/second');
```

Step 3: Go Back

dart

CopyEdit

```
Navigator.pop(context);
```

---

3. Passing Data Between Screens

We can pass data when navigating to another screen.

Sending Data to Next Screen

```
Navigator.push(
  context,
  MaterialPageRoute(builder: (context) => SecondScreen(data: 'Hello!')),
);
```

Receiving Data in Second Screen

```
class SecondScreen extends StatelessWidget {
  final String data;

  SecondScreen({required this.data});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Center(child: Text(data)),
    );
  }
}
```

Second Screen

This is the Second Screen

Back to Main Screen

Main Screen

Welcome to the Main Screen

Go to Second Screen

Conclusion

Flutter's navigation system allows smooth transitions between screens using Navigator.push(), Navigator.pop(), and named routes. Proper routing enhances app structure and maintainability. Learning these concepts is essential for developing multi-screen applications.